

MEDIATEK

MT8676 Android System User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Chao Zhang	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	4
List of Tables	4
1 System	5
1.1 Build System	5
1.1.1 Build Environment	5
1.1.2 Sync Code	5
1.1.3 Build Command	6
1.2 System Overview	6
1.2.1 Brief Introduction	6
1.2.2 Abbreviations	6
1.3 Architecture/Process Overview	7
1.3.1 MT8676 Android Software Layers	7
1.3.2 Bootup	8
1.3.3 Shutdown	14
1.3.4 Android Bootup Log Analysis	15
1.4 Configuration/Customization Guideline	19
1.4.1 System Related Configurations	19
1.5 Frequently Asked Questions/Troubleshooting	21
1.5.1 Logs Provision	21
1.5.2 MT8676 DB Analysis Guide	21
1.5.3 Memory Leak	30
1.5.4 WDT Brief Introduction	31
1.5.5 Performance Mode	32
1.5.6 CPU Hotplug	33
1.5.7 Systrace Capture	34
1.5.8 System Reference FAQ	34
Exhibit 1 Terms and Conditions	35

List of Figures

Figure 1-1. MT8676 Android software layers	7
Figure 1-2. Bootup architecture	8
Figure 1-3. Main work of Boot Rom	9
Figure 1-4. Main work of Bootloader	10
Figure 1-5. Main work of kernel C phase	11
Figure 1-6. MT8676 shutdown flow	14
Figure 1-7. DB category	22
Figure 1-8. Exception classification	23
Figure 1-9. GAT tool UI	24
Figure 1-10. DBViewer UI	24
Figure 1-11. Generated file after parsing	24
Figure 1-12. Files generated after decompression	25
Figure 1-13. USBNET causing KE	25
Figure 1-14. SWT DB files generated after decompression	26
Figure 1-15. system_server causing SWT	26
Figure 1-16. NE DB file gen	27
Figure 1-17. Systemui causing NE	27
Figure 1-18. JE DB file generated after decompression	28
Figure 1-19. Animation causing system_server crash and leading to JE	29
Figure 1-20. MD ASSERT triggered	29
Figure 1-21. ANR occurs in system UI	30
Figure 1-22. How to quickly locate ANR issues through logs	30
Figure 1-23. cat /proc/meminfo log	31
Figure 1-24. Kernel memory leak log	31

List of Tables

Table 1-1. Abbreviations	6
Table 1-2. System issue log map (M: must, O: optional, C: conditional, X: no need)	21
Table 1-3. KE log map	26
Table 1-4. NE log map	28
Table 1-5. ANR issue log map	30

1 System

1.1 Build System

1.1.1 Build Environment

Item	Requirement
Disk Space	At least 250GB
PC Memory	At least 24GB
Version	Ubuntu 18.04 (Recommended)
Tools which should be installed for compilation	Refer to http://source.android.com/source/initializing.html
Git	>1.9.1
Shell	>4.4.1 (Ubuntu18.04 default built-in version) (Recommended)
Perl	>5.26.1 (Ubuntu18.04 default built-in version) (Recommended)
Python	Python 2.7.17&Python 3.6.9 (Ubuntu18.04 default built-in version) (Recommended)
GNU Make	>4.1 (Ubuntu18.04 default built-in version) (Recommended)

1.1.2 Sync Code

Step1: Make a new directory to save the download.

cmd: mkdir <working folder> && cd <working folder>

Step2: Codebase initialization

cmd: repo init -u https://git01.mediatek.com/alps_release/platform/manifest -b <branch> -m <manifest> --no-repo-verify

<branch>: branch name

<manifest>: Specific branch and version

Scenario 1: init latest code

```
$repo init -u https://git01.mediatek.com/alps_release/platform/manifest -b $company -m $(release_branch)-default.xml --no-repo-verify
```

Scenario 2: init specific version

```
$repo init -u https://git01.mediatek.com/alps_release/platform/manifest -b $company -m $(release_tag).xml --no-repo-verify
```

Step3: Code Sync

Scenario 1: Sync full codebase

```
$repo sync -c -f -j8 --no-repo-verify --optimized-fetch
```

Scenario 2: Sync single repo

```
$repo sync <repository name> /*<repository name> can be repository name or repository path */
```

Example: \$repo sync device/common

Scenario 3: Sync multiple repos

```
$repo sync <repository name1 repository name2> /*repositpryl repository2 separated by space */
```

Example: \$repo sync platform/art platform/abi/cpp

1.1.3 Build Command

MT8676 uses LD2.0 build. You can refer to MOL: <https://online.mediatek.com/apps/quickstart/QS00266>.

Build command:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py -run full_${Project}-${mode}
```

For example:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py -run full_auto8676p1_64_bsp-userdebug
```

Output: Images will be created in out/target/product/\${project}/merged.

LD2.0 will create out_sys & out_hal & out_krn & out to store the images of each layer after build.

For detailed information, refer to: <https://online.mediatek.com/apps/quickstart/QS00178>.

Query build command:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_${Project}-${mode}
```

1.2 System Overview

1.2.1 Brief Introduction

This chapter mainly introduces MT8676 system related content and how to deal with common system problems.

1.2.2 Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
AED	Android Exception Daemon
JE	Java Exception
NE	Native Exception
KE	Kernel Exception
EE	External Exception
ANR	Application Not Response
SWT	Software Watchdog Timeout

1.3 Architecture/Process Overview

1.3.1 MT8676 Android Software Layers

MT8676 Android software layers diagram is shown in [Figure 1-1](#) (Zoom in to see clearly).

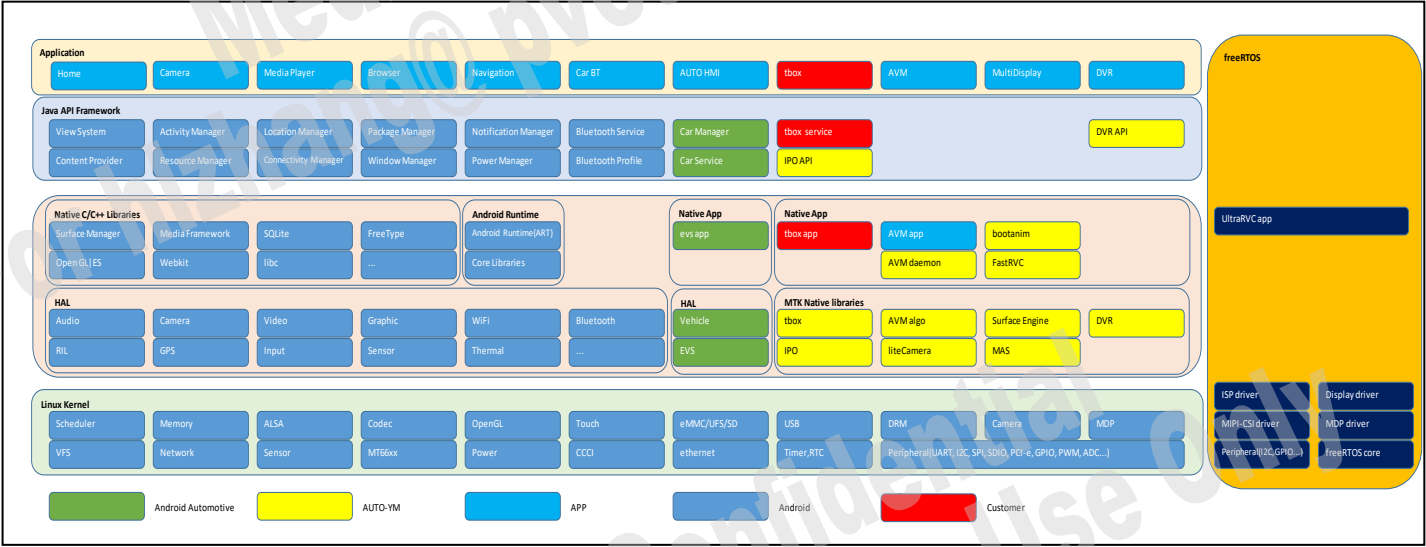


Figure 1-1. MT8676 Android software layers

1.3.2 Bootup

1.3.2.1 Bootup Architecture

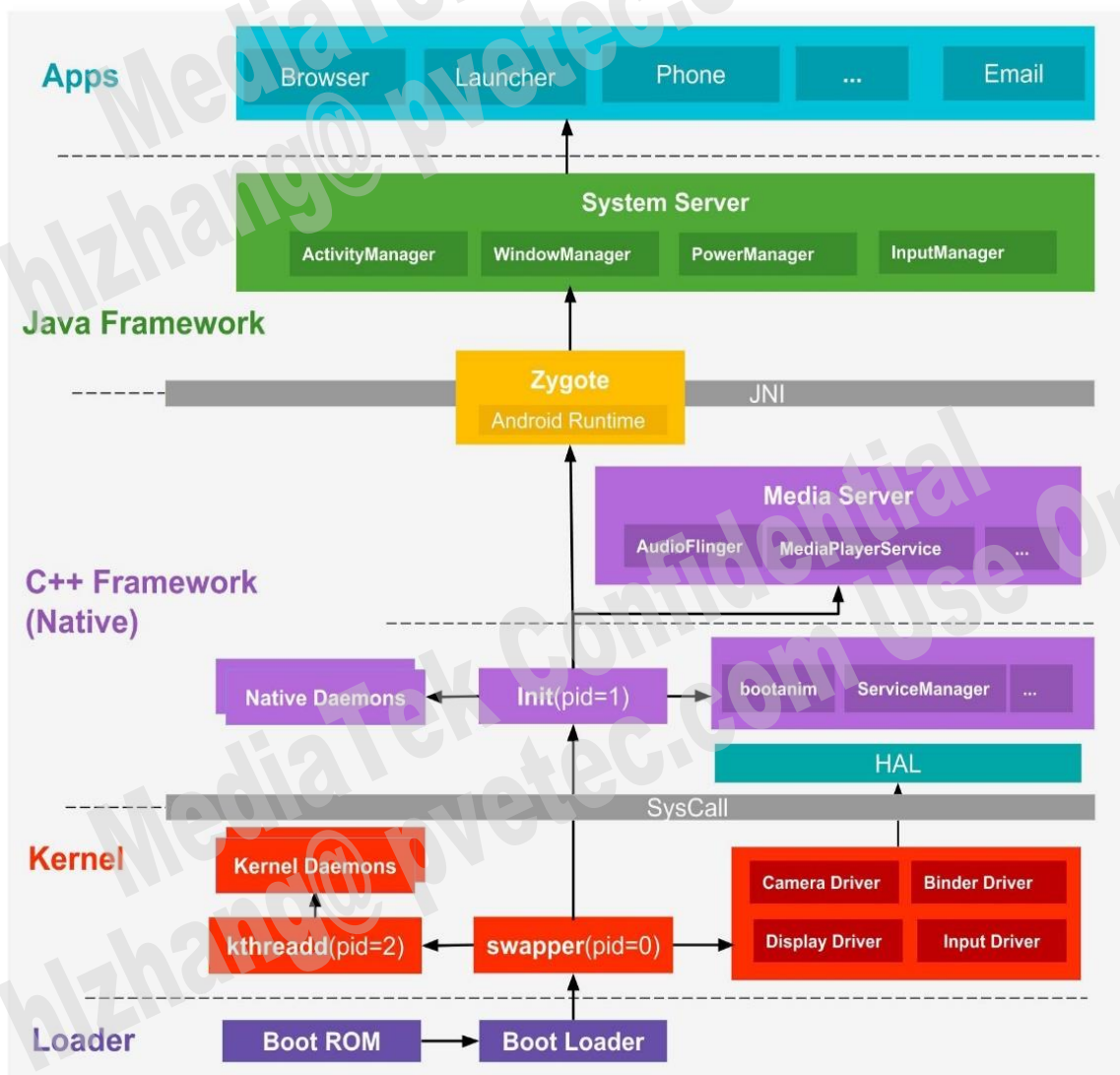


Figure 1-2. Bootup architecture

- Step1: Once the system is powered on, the bootroom code will be executed firstly, then system loads the bootloader from the UFS into the SRAM, and initializes the DRAM, and boots up the Linux kernel. Boot Loader is divided into preloader and LK. The preloader runs in SDRAM, but LK runs in DRAM, and LK will load the kernel image.
- Step2: Idle process will be start firstly, which is used to initialize process management and memory management and load the driver module; then fork the kthreadd process, which is the father process of all Linux kernel processes.
- Step3: After the module drivers are loaded, the drivers will interact with the HAL layer and start init process.
- Step4: After the init process is started, various user daemons will be started, and some important services such as servicemanager (binder service manager) will be also started. At the same time, the corresponding configuration file init.rc will be parsed and Zygote will be forked.

- Step5: The Zygote is a process between the native layer and the Java layer. It will load the virtual machine JVM, register the JNI function, open the channel from the native to the Java layer, and start the systemserver (responsible for the entire Java framework) and the first app process (launcher).

1.3.2.2 Boot ROM

1.3.2.2.1 Brief Introduction

Boot Rom is a hardware device (similar to SRAM/DRAM), which contains an unmodifiable and solidified program bootcode that comes with the factory.

After powering on, Boot Rom starts to execute the internal bootcode that has been set in advance, loads the boot program preloader into SRAM and then executes it.

1.3.2.2.2 Main Work

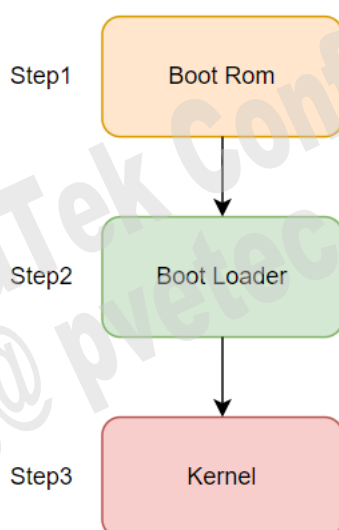


Figure 1-3. Main work of Boot Rom

1. Load the boot program preloader into SRAM and then execute it.
2. When no valid image or code is detected, Boot Rom boots into download mode.
3. Perform some verification work.
4. Hardware initialization, serial port, flash, etc.

1.3.2.3 Bootloader

1.3.2.3.1 Brief Introduction

The bootloader is divided into preloader and LK (Little Kernel is an open source bootlader project, but it only supports Arm and x86 platforms. Its notable feature is the implementation of a simple thread mechanism).

The preloader is stored in Flash and runs in SRAM (SRAM does not need to be initialized). The preloader can be directly loaded into SRAM by bootrom and run. Its main function is to initialize DRAM which LK runs in. The main function of LK is to load the kernel into DRAM.

1.3.2.3.2 Main Work

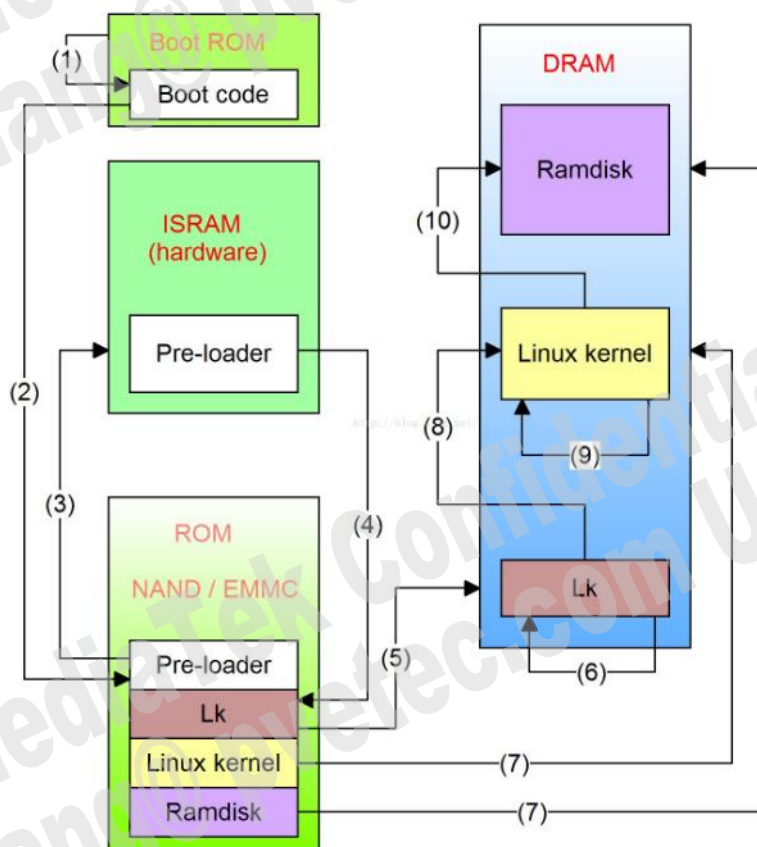


Figure 1-4. Main work of Bootloader

1. After powering on, the BootCode will be executed first.
2. BootCode locates the preloader in Flash.
3. Load the preloader from Flash into SRAM (0x00200F10) and start to initialize the external DRAM.
4. The preloader locates LK in the Flash.
5. Load LK from Flash to external DRAM for operation.
6. LK initializes itself.
7. Decompress boot.image/vendor_boot.img into kernel and ramdisk by LK, and then load them from Flash into DRAM.
8. Initialize kernel by LK.
9. Execute part of the kernel program (fork out the init process after the kernel initialization is completed).
10. Boot up the init program in the ramdisk, enter the user space for initialization, and then the init process forks out the Zygote process until the entire Android startup is completed.

1.3.2.4 Kernel

1.3.2.4.1 Brief Introduction

The kernel is the core component of the operating system. It is responsible for managing computer hardware and software resources and providing basic services for the operating system and applications. The kernel is the first load of the operating system and runs in DRAM as the core of the entire operating system. It controls all system resources, such as CPU, Memory, input/output, file system, etc. The kernel provides underlying services and interfaces for applications to call and to use.

1.3.2.4.2 Main Work

Kernel assembly phase (started by ENTRY(_stext)):

1. Set to SVC mode and turn off all interrupts.
2. Get CPU ID.
3. Verify dtb.
4. Create page table entries.
5. Configure the r13 register which is set for jumping to phase C after opening the MMU.
6. Enable MMU.
7. Jump to start_kernel and enter phase C.

Kernel C phase (started by kernel_init)

Complete the initialization work related to the hardware platform and execute the init process after the initialization is completed.

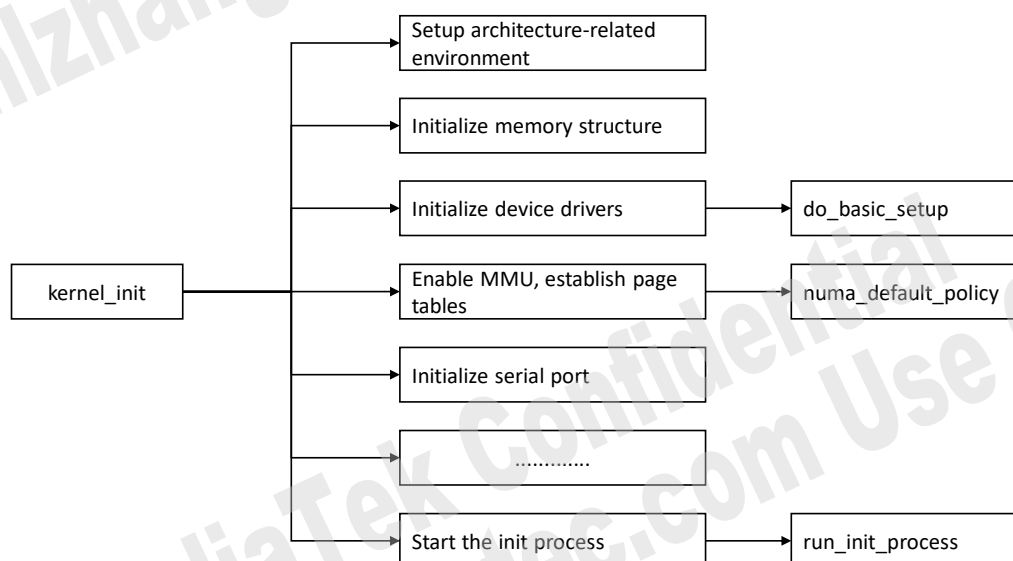


Figure 1-5. Main work of kernel C phase

1.3.2.5 Init Process

1.3.2.5.1 Brief Introduction

The init process is the first process in user space in the Linux OS, which PID is 1. The init process will start after the kernel started and will read the relevant configuration in `init.rc` to start other related processes.

1.3.2.5.2 Main Work

- Analyze the `init.rc` script file and execute the corresponding function according to the file content.
- When some critical process dies, init process will restart the died process.
- Provide property services of Android OS.

1.3.2.5.3 Code Introduction

The entry of Init is located in `/system/core/init/init.c`.

1.3.2.6 Zygote

1.3.2.6.1 Brief Introduction

When an Android device boots up, the system first starts the Zygote process. The Zygote process will preload and initialize some commonly used system libraries and resource files, which can speed up the startup of other applications because these resources have been loaded into DRAM.

1.3.2.6.2 Main Work

The following five tasks are completed in the main function of the `ZygoteInit` class.

1. Register a listening socket for the Zygote process using `registerZygoteSocket()`.
2. The function `preload()` loads commonly used Java classes and system resources.
3. The function `startSystemServer()` is used to initiate the `SystemServer` process.
4. The function `runSelectLoopMode()` enters the loop monitoring mode.
5. Once process exits, the socket listener should be closed using the `closeServerSocket()` function.

1.3.2.6.3 Code Introduction

The entry of Zygote is located in `kernel/sched/fork.c`.

1.3.2.7 SystemServer

1.3.2.7.1 Brief Introduction

The SystemServer is created by the `main` function of `ZogoteInit`, it will execute the `main` function of SystemServer after calling `caller.run`.

1.3.2.7.2 Main Work

- The SystemServer manages many core services and functions of the Android OS, such as ActivityManager Service, Content Provider Service, Package Manager Service, Notification Manager Service and more. They provide essential infrastructure for applications to run and interact.
- Managing system resources: The System Server manages system resources through the Resource Manager, which includes memory, processes, threads, file systems, networks and more. The Resource Manager can allocate and reclaim system resources to ensure system stability and security.
- Providing system-level settings and configurations: The System Server provides system-level settings and configurations such as network settings, volume settings, time settings, language settings. These settings are at the system level and impact the overall performance and operation.

1.3.2.7.3 Code Introduction

The entry of SystemServer is located at

`frameworks/base/services/java/com/android/server/SystemServer.java`.

1.3.3 Shutdown

1.3.3.1 Shutdown Flow Chart

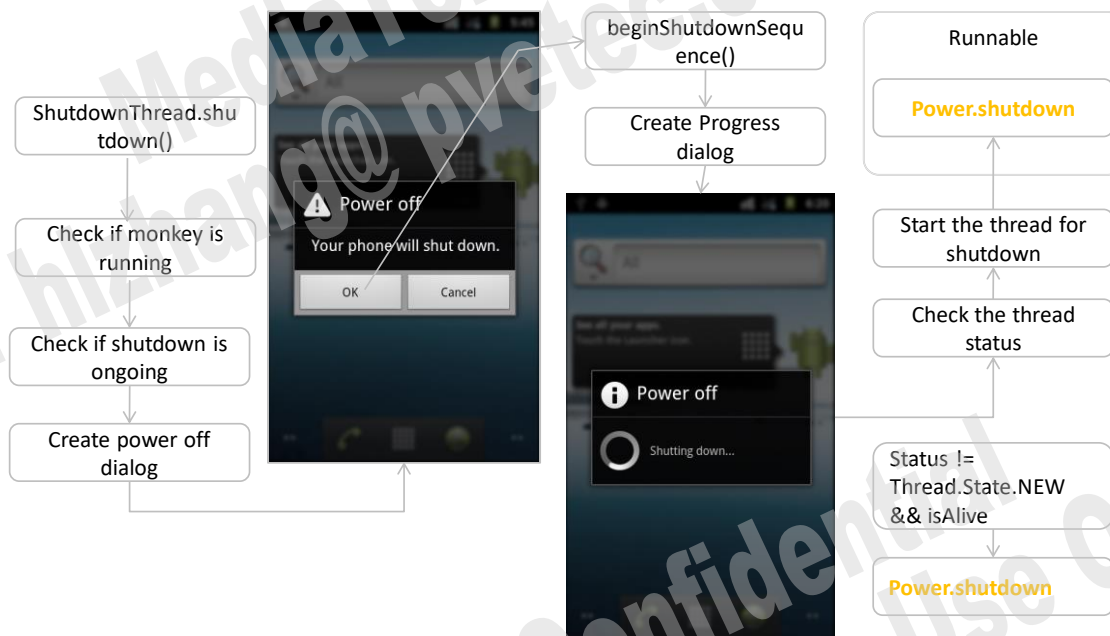


Figure 1-6. MT8676 shutdown flow

- Invoke `ShutdownThread.shutdown()`.
 1. Obtain user shutdown behavior.
 2. Register shutdown broadcast.
 3. Create a shutdown dialog.
 4. Execute a series of shutdown procedures.
- When the power off dialog appears, if the shutdown is confirmed by the user, `beginShutdownSequence` will be initiated to start the shutdown process.
 1. Display the shutdown progress dialog box.
 2. Start the `ShutdownThread` and execute the run method.

1.3.3.2 ShutdownThread Main Function

1. Broadcast an orderly shutdown sequence.
2. Measure the time it takes to close certain projects.
3. Disable the `ActivityManager`.
4. `PackageManager` is to be closed.
5. Turn off the radios.
6. Complete the shutdown of the `SystemService`.
7. Initialize and determine whether to shut down or restart.

If the IPO function is available, the device will enter IPO and then enter sleep mode.

1.3.4 Android Bootup Log Analysis

```
[14:40:10:408] Pll init start... //The first log entry of the program indicates that
the program has started and entered the perloader.
...
[14:40:10:444] Pll init Done!!
[14:40:10:444] #T#PLL=24
[14:40:10:445] #T#GPIO=0
[14:40:10:445] [RGU] rst from: kernel
[14:40:10:445] drm_latch_en: MTK_DRM_LATCH_EN(95027EF0)
[14:40:10:445] [RGU] STA from reg: 0x40000000 //The RGU printing can be
monitored for any abnormal system reboots.
[14:40:10:446] [RGU] MODE: 0x25
[14:40:10:446] [RGU] STA: 0x40000000
[14:40:10:446] [RGU] LENGTH: 0x1FFFE0
[14:40:10:446] [RGU] INTERVAL: 0xFFFF
[14:40:10:447] [RGU] SWSYSRST: 0x0
[14:40:10:447] [RGU] LATCH_CTL: 0xF21E79
[14:40:10:447] [RGU] LATCH_CTL2: 0x0
[14:40:10:447] [RGU] NONRST_REG: 0x0
[14:40:10:448] [RGU] NONRST_REG2: 0x2C002000
[14:40:10:448] [RGU] REQ_MODE: 0xFD00E3
[14:40:10:448] [RGU] REQ_IRQ_EN: 0x7D0065
[14:40:10:448] [RGU] DEBUG_CTL: 0x0
[14:40:10:448] [RGU] parse g_rgu_status: 2 (0x2)
[14:40:10:449] [RGU] Set NONRST_REG to 0x40000000
[14:40:10:449] [RGU] mtk_wdt_mode_config mode value=34, tmp:22000034
...
[14:40:10:541] [PMIC]PONSTS[0xC]=0x0 //Printing the PMIC registers allows monitoring
for any abnormal behaviors related to the PMIC.
[14:40:10:541] [PMIC]POFFSTS[0xD]=0x400
[14:40:10:541] [PMIC]POFFSTS2[0xF]=0x0
[14:40:10:542] [PMIC]POFFSTS4_5[0x11]=0xFFFF
[14:40:10:542] [PMIC]POFFSTS6[0x13]=0xF
[14:40:10:542] [PMIC]VR_SDN_MODE0_1[0x214]=0x0
[14:40:10:542] [PMIC]VR_SDN_MODE2_3[0x216]=0x0
[14:40:10:543] [PMIC]VR_SDN_MODE4_5[0x218]=0x0
[14:40:10:543] [PMIC]VR_SDN_MODE6_7[0x21A]=0x0
[14:40:10:547] [PMIC]STS_THR_LOC[0x10]=0x0
[14:40:10:547] [PMIC]STRUP_CON4[0xA1A]=0x0
[14:40:10:548] [PMIC]STRUP_CON12[0xA0F]=0x1
[14:40:10:548] [PMIC]WDTRSTB[0x139]=0x4
[14:40:10:548] [pmic_check_rst] AP Watchdog
...
[14:40:12:051] [Calibration Summary] Frequency 3750 //DRAM calibration at each
frequency point.
[14:40:12:051] CH 0, Rank 0
[14:40:12:051] SW Impedance : PASS
[14:40:12:051] 8 Phase : FAST K
[14:40:12:051] DUTY Scan : FAST K
```

```

[14:40:12:051] ZQ Calibration      : PASS
[14:40:12:052] Jitter Meter       : NO K
[14:40:12:052] CBT Training        : NO K
[14:40:12:052] Write leveling(PI)    : FAST K
[14:40:12:052] Write leveling(DLY)   : FAST K
[14:40:12:052] DUTY CYCLE MONITOR  : FAST K
[14:40:12:052] RX DQS gating      : FAST K
[14:40:12:052] RX OFFC         : NO K
[14:40:12:053] RX DQ/DQS(RDDQC)     : FAST K
[14:40:12:053] RX DQ/DQS(RDDQC_DQM_ONLY) : FAST K
[14:40:12:053] TX DQ/DQS        : PASS
[14:40:12:053] RX DATLAT       : FAST K
[14:40:12:053] RX DQ/DQS(Engine)   : FAST K
[14:40:12:053] RX RDQS DCA      : FAST K
[14:40:12:054] RX RDQS DCC      : FAST K
[14:40:12:054] TX OE         : NO K
[14:40:12:054] All Pass.

...
[14:40:12:892] [BLDR] Starting tool handshake.      //The USB and Flashtool USB perform a
handshake, and if successful, the DA will be loaded to enter download mode.
[14:40:15:971] #T#UART handshake init=1

...
[14:40:16:098] [PART] load "lk_a" from 0x000000003B8DB9C0 (dev) to 0x78000000 (mem)
[SUCCESS]      //The preloader loads the relevant images into the DRAM.
[14:40:16:098] [PART] load speed: 155765KB/s, 638016 bytes, 4ms
[14:40:16:098] [PART] img vfy...[SEC] img auth ok

...
[14:40:16:418] welcome to lk      //Enter the lk BL2_EXT.
[14:40:16:418]
[14:40:16:418] lk variant: BL2_EXT

...
[14:40:19:081] Say Hello from EL2!
[14:40:19:082]
[14:40:19:082] MIDR: 411fd461, MPIDR: 81000000
[14:40:19:082] CPU Core: 0!
[14:40:19:082] GZ Platform: MTK_ARMV8
[14:40:19:082] Memory base: 0xbe5400000, size: 0x2a00000
[14:40:19:083] lk_init 0x38082728-0x38082ba8=1152
[14:40:19:083] 48600000 48600000 8000 11001000 in lk_main
[14:40:19:083] ====> GZ (0)[ 0.002364]Version of GZ-CORE: GZ_CORE_hypervisor:
3.2.0.039.UOMP1, Built: 11:53:14 Sep 6 2023
[14:40:19:083] ====> GZ (0)[ 0.003564]platform_init_mmu_mappings membase: 0xbe5400000
[14:40:19:084] ====> GZ (0)[ 0.004372]platform_init_mmu_mappings memsize: 0x2a00000
[14:40:19:084] ====> GZ (0)[ 0.005158]platform_init_mmu_mappings kernel offset:
0x38000000
[14:40:19:084] ====> GZ (0)[ 0.006019]platform_init_mmu_mappings ram phys: 0xc1d400000
[14:40:19:085] ====> GZ (0)[ 0.006838]platform_init_mmu_mappings ram size: 2a00000
[14:40:19:085] ====> GZ (0)[ 0.007740]INIT: cpu 0, calling hook 0x3800f0f0
(libvmm_early) at level 0x10000, flags 0x1
[14:40:19:085]
[14:40:19:085] welcome to lk/MP...      //Enter GZ

...
[14:40:20:982] welcome to lk      // Enter LK
[14:40:20:995]

```

```

[14:40:20:995] lk variant: BL33
[14:40:20:995] boot args 0x48600000 0x0 0x0 0x0
[14:40:20:995] version:
[14:40:20:995]     arch:    arm64
[14:40:20:995]     platform: mediatek
[14:40:20:995]     target:   auto8676p1_64_bsp
[14:40:20:995]     project:  auto8676p1_64_bsp
[14:40:20:997]     buildid: d130cca93e_202407032103
...
[14:40:25:919] [PROFILE] ::: hyp_unmap2() takes 1376 ms
[14:40:25:925] [PROFILE] ::: name: pl_t, time: 6106 ms
[14:40:25:925] [PROFILE] ::: name: logo_t, time: 1533 ms
[14:40:25:925] [PROFILE] ::: name: lk_t, time: 5048 ms
[14:40:25:929] [PROFILE] ::: name: bl2_ext_t, time: 1842 ms
[14:40:25:929] [PROFILE] ::: name: tfa_t, time: 644 ms
[14:40:25:929] [PROFILE] ::: name: sec_os_t, time: 0 ms
[14:40:25:929] [PROFILE] ::: name: gz_t, time: 1860 ms
[14:40:25:934] boot_linux_fdt:543: booting linux @ 0x40000000, ramdisk @ 0x66f00000
(31301139) dtb @ 0x47c80000 (487064)
[14:40:25:934] boot_linux_fdt:544: lk boot mode = 0
[14:40:25:934] boot_linux_fdt:546: lk finished --> jump to linux kernel 64Bit //LK
jumps to kernel
...
[14:40:26:122] [ 0.000000][ T0] swapper: [name:setup&]Bootting Linux on physical CPU
0x0000000000 [0x411fd461] //The first line is printed when the kernel starts.
[14:40:26:122] [ 0.000000][ T0] swapper: [name:main&]Linux version 6.1.78-android14-
11-maybe-dirty (build-user@build-host) (Android (10087095, +pgo, +bolt, +lto, -mlgo, based
on r487747c) clang version 17.0.2 (https://android.googlesource.com/toolchain/llvm-project
d9f89f4d16663d5012e5c09495f3b30ece3d2362), LLD 17.0.2) #1 SMP PREEMPT Thu Jan 1 00:00:00
UTC 1970
[14:40:26:124] [ 0.000000][ T0] swapper: [name:setup&]Machine model: MT8676
[14:40:26:124] [ 0.000000][ T0] swapper: [name:stackdepot&]Stack Depot is disabled
[14:40:26:124] [ 0.000000][ T0] swapper: [name:kvm&]KVM is not available. Ignoring
kvm-arm.mode
[14:40:26:124] [ 0.000000][ T0] swapper: earlycon: [name:earlycon&]uart8250 at
MMIO32 0x0000000011001000 (options '')
...
[14:40:30:574] [ 3.387865][T700001] init: mtk-smi-larb 1a02c000.smi-larb25:
[name:mtk_smi&]Succeed to get smi-comm dev for mmqos //init load ramdisk ko
[14:40:30:574] [ 3.389412][T700075] kworker/7:1: [name:mtk_vmm_spm&][ISPDVFS]
regulator_event_notify(): VMM regulator before disable, ver = 3
[14:40:30:575] [ 3.389412][T700075]
[14:40:30:575] [ 3.391773][T700001] init: [name:mtk_vmm_spm&][ISPDVFS]
regulator_event_notify(): VMM regulator enable done, ver = 3
[14:40:30:576] [ 3.391773][T700001]
[14:40:30:576] [ 3.393960][T700001] init: mtk-smi-larb 1a02d000.smi-larb26:
[name:mtk_smi&]Succeed to get smi-comm dev for mmqos
[14:40:30:576] [ 3.395507][T700075] kworker/7:1: [name:mtk_vmm_spm&][ISPDVFS]
regulator_event_notify(): VMM regulator before disable,
...
[14:40:36:929] [ 8.972200][T1400001] init: init 23: Parsing file
/system/etc/init/hw/init.rc... //init parse rc file
[14:40:36:937] [ 8.974057][T1600001] init: init 23: Added '/init.envIRON.rc' to import
list

```

```

[14:40:36:939] [ 8.974965][T1600001] init: init 23: Added
'/system/etc/init/hw/init.usb.rc' to import list
[14:40:36:945] [ 8.976021][T1600001] init: init 23: Added '/init.mt8676.rc' to import
list
[14:40:36:965] [ 8.976902][T1600001] init: init 23: Added
'/vendor/etc/init/hw/init.mt8676.rc' to import list
[14:40:36:965] [ 8.978016][T1600001] init: init 23: Added
'/system/etc/init/hw/init.usb.configfs.rc' to import list
[14:40:36:965] [ 8.979172][T1600001] init: init 23: Added
'/system/etc/init/hw/init.zygote64.rc' to import list
[14:40:36:975] [ 8.980525][T1600001] init: init 23: Added
'/system/etc/init/hw/init.boringssl.zygote64.rc' to import list
[14:40:36:976] [ 8.982166][T1600001] init: init 23: Parsing file /init.environ.rc...
[14:40:36:977] [ 8.983054][T1600001] init: init 5: Parsing file
/system/etc/init/hw/init.usb.rc...
[14:40:36:978] [ 8.984895][T1600001] init: init 5: Parsing file /init.mt8676.rc...
[14:40:36:978] [ 8.985735][T1600001] init: init 5: Unable to read config file
'/init.mt8676.rc': open() failed: No such file or directory
...
[14:40:42:042] [ 14.402919][T1400415] modprobe: mt6338-efuse mt6338-efuse:
[name:nvmem_mt6338_efuse&]EFUSE[58]=0x83 //load
vendor ko
[14:40:42:042] [ 14.404272][T1400415] modprobe: mt6338-sound mt6338-sound:
[name:snd_soc_mt6338&]mt6338_codec_init_reg()
[14:40:42:042] [ 14.417048][T1400415] modprobe: snd-scp-ultra snd-scp-ultra:
[name:snd_soc_mtk_scp_ultra&]mtk_scp_ultra_pcm_new()
[14:40:42:043] [ 14.418416][T1400415] modprobe: snd-scp-ultra snd-scp-ultra:
[name:snd_soc_mtk_scp_ultra&]mtk_scp_ultra_reserved_dram_init(), sce reserve mem
pa=0x8f835700, va=0xfffffff018835700, size=0x1a000
[14:40:42:043] [ 14.420562][T1400415] modprobe: snd-scp-ultra snd-scp-ultra:
[name:snd_soc_mtk_scp_ultra&]mtk_scp_ultra_reserved_dram_init(), dump pa=0x8f835700,
va=0xfffffff018835700, size=0x1a000
...
[14:40:47:477] [ 20.251333][T1100322] init: init 29: [20203][1012]Wait for property
'vendor.all.modules.ready=1' took 6885ms
[14:40:47:477] [ 20.252841][T1700324] init: [name:bootprof&]BOOTPROF:
20252.839124:modprobe: Load_Module_DONE //ko load complete
[14:40:47:477] [ 20.254094][T1700001] init: init 25: [20211][5676]Lastest epoll wait
took 5676ms
[14:40:47:485] [ 20.255294][T1700001] init: init 25: [20213][0]Command 'write
/proc/sys/vm/watermark_boost_factor 0' action=post-fs-data
(/vendor/etc/init/hw/init.mt8676.rc:178) took 0ms and failed: Unable to write to file
'/proc/sys/vm/watermark_boost_factor': open() failed: Permission denied
[14:40:47:488] [ 20.258432][T1700001] init: init 25: [20213][0]Service 'insmod_sh' (pid
387) exited with status 0 oneshot service took 10.190000 seconds in background
[14:40:47:488] [ 20.260124][T1700001] init: init 25: [20213][0]Sending signal 9 to
service 'insmod_sh' (pid 387) process group...
...
[14:40:48:412] [ 21.214884][T1500324] init: [name:bootprof&]BOOTPROF:
21214.882203:INIT:zygote-start //start zygote process
[14:40:48:413] [ 21.216010][T1500001] init: init 25: [21174][0]processing action
(firmware_mounts_complete) from (/system/etc/init/hw/init.rc:521)
[14:40:48:413] [ 21.217542][T1500001] init: init 25: [21175][0]processing action (early-
boot) from (/vendor/etc/init/hw/init.mt8676.rc:751)

```

```
[14:40:48:413] [ 21.219106][T1500324] init: [name:bootprof&]BOOTPROF:
21219.105049:INIT:early-boot //enter early boot stage,config some basic services and
parameters.
...
[14:40:55:318] [ 28.131865][T1700324] init: [name:bootprof&]BOOTPROF: 28131.864681:
OFF (KO:356) //boot complete
```

1.4 Configuration/Customization Guideline

1.4.1 System Related Configurations

1.4.1.1 Fastboot Configuration

Preloader:

```
vendor/mediatek/proprietary/bootable/bootloader/preloader/custom/auto8676p1_64_bsp/auto867
6p1_64_bsp.mk

+ MTK_SEC_USBDL=ATTR_SUSBDL_DISABLE
+ MTK_SEC_BOOT=ATTR_SBOOT_DISABLE
---
- MTK_SEC_USBDL=ATTR_SUSBDL_ONLY_ENABLE_ON_SCHIP
- MTK_SEC_BOOT=ATTR_SBOOT_ENABLE
15c15
+ MTK_SECURITY_SW_SUPPORT=no
---
- MTK_SECURITY_SW_SUPPORT=yes
58,71c58
+ MTK_CPU_BOOST = yes
+ MTK_REDUCE_DRAM_FREQ = yes
+ ifeq ("$(TARGET_BUILD_VARIANT)","user")
+     CFG_USB_TOOL_HANDSHAKE :=0
+     CFG_LOG_LEVEL :=0
+     CFG_LOG_STORE_SUPPORT := 0
+     CFG_ATF_LOG_SUPPORT := 0
+     CFG_DOE_CONFIG_ENV_SUPPORT:= 0
+ endif
+ ifeq ("$(TARGET_BUILD_VARIANT)","userdebug")
+     CFG_USB_TOOL_HANDSHAKE :=1
+     CFG_LOG_LEVEL :=2
+ endif
```

LK2:

```
vendor/mediatek/proprietary/bootable/bootloader/lk2/project/auto8676p1_64_bsp.mk

27c27
+ MTK_LCM_LIST_SUPPORT="jd9365da_wxga_dsi_vdo"
---
- MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo jd9365da_wxga_dsi_vdo"
30c30
+ MTK_SECURITY_SW_SUPPORT := no
```

```

---
- MTK_SECURITY_SW_SUPPORT := yes
34,44d33
+ MTK_DISABLE_GZ := yes
+ MTK_LK_NO_DISPLAY := yes
+ ifeq ("$(TARGET_BUILD_VARIANT)","user")
+ MTK_LK_LOG_DISABLE := yes
+ LK_WITH_SMP :=yes
+ MTK_DM_VERITY_OFF := yes
+ override DEBUG := 0
+ endif
+ ifeq ("$(TARGET_BUILD_VARIANT)","userdebug")
+ override DEBUG := 1
+ endif

```

TF2.8:

```

vendor/mediatek/proprietary/trustzone/tf-a-
2.8/plat/mediatek/project/auto8676p1_64_bsp/platform.mk

MTK_SOC := mt6897
+CONFIG_MTK_GZ := n
+CONFIG_MTK_GZ_SECIO := n
+CONFIG_MTK_GZ_SMC := n
+LOG_LEVEL := 0
include plat/mediatek/$(MTK_SOC)/platform_makefile.mk

```

Kernel Config:

```
kernel/kernel_device_modules-6.1/arch/arm64/configs/auto8676p1_64_defconfig
```

You need to set configurations as below:

```

CONFIG_ARM_FFA_TRANSPORT=n
CONFIG_MTK_GZ_KREE=n
CONFIG_MTK_DISABLE_GZ=y
CONFIG_MTK_DRAM_LOG_STORE=n
CONFIG_MTK_SENSOR_FAST_PROBE=y
# CONFIG_MTK_DRAM_LOG_STORE_ADDR is not set
# CONFIG_MTK_DRAM_LOG_STORE_SIZE is not set
# CONFIG_MTK_SECURITY_SW_SUPPORT is not set

```

Ko Table:

```
device/mediateksample/auto8676p1_64_bsp/ko_order_table.csv
```

According to the following method, delete or add ko.

```

delete,mkp.ko//Delete the row corresponding to mkp.ko in ko_order_table.csv.
delete,ffa_v10.ko
delete,cmdq-sec-drv.ko
delete,mcDrvModule-ffa.ko
delete,gz_trusty_mod.ko
delete,gz_ipc_mod.ko
delete,gz_irq_mod.ko
delete,gz_virtio_mod.ko

```

```

delete,gz_tz_system.ko
delete,gz_main_mod.ko
delete,gz_log_mod.ko
delete,tmem_ffa.ko
delete,mtk_sec_heap.ko
delete,sapu.ko
delete,log_store.ko
delete,sec.ko
delete,phy-mtk-pcie.ko
delete,pcie-mediatek-gen3.ko
add,teeperf.ko,isee-ffa.ko,mcDrvModule.ko,../../kernel_device_modules-
6.1/drivers/tee/gud/600/MobiCoreDriver/mcDrvModule.ko,ramdisk,Y,Y,user/userdebug/eng
// Insert mcDrvModule.ko after teeperf.ko and before isee-ffa.ko.
add,,cfg80211.ko,phy-mtk-pcie.ko,../../kernel_device_modules-6.1/drivers/phy/mediatek/phy-
mtk-pcie.ko,vendor,Y,Y,user/userdebug/eng
//Insert phy-mtk-pcie.ko before cfg80211.ko.
add,,cfg80211.ko,pcie-mediatek-gen3.ko,../../kernel_device_modules-
6.1/drivers/pci/controller/pcie-mediatek-gen3.ko,vendor,Y,Y,user/userdebug/eng
//Insert the pcie-MediaTek-gen3.ko module before the cfg80211.ko module.

```

1.5 Frequently Asked Questions/Troubleshooting

1.5.1 Logs Provision

Table 1-2. System issue log map (M: must, O: optional, C: conditional, X: no need)

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
Bootup	M	M	O	O	X	X	X
NE	M	X	M	M	X	X	X
KE	M	C	M	M	X	X	X
SWT	M	C	M	C	X	X	X
ANR	M	X	M	O	C	C	C
JE	M	X	M	X	X	X	X
IPO	M	M	O	O	X	X	X
Performance	M	M	O	X	M	M	C

1.5.2 MT8676 DB Analysis Guide

1.5.2.1 AEE

- Android Exception Engine (AEE), the mechanism for capturing Log developed by MediaTek itself.
- The mechanism for capturing abnormal information when the system encounters an exception is called AEE. Abnormal information occurring before and after the exception can be captured in the form of log for a period of time and then packaged and compressed into a DB file, which is saved to the corresponding location on UFS.

1.5.2.2 DB Storage Path

The AEE database storage path:

/data/aee_exp
/data/vendor/aee_exp

1.5.2.3 DB Category

JE	• Java Exception
NE	• Native Exception
KE	• Kernel Exception
EE	• External Exception
ANR	• Application Not Response
SWT	• Software Watchdog Timeout

Figure 1-7. DB category

1.5.2.4 Exception Classification

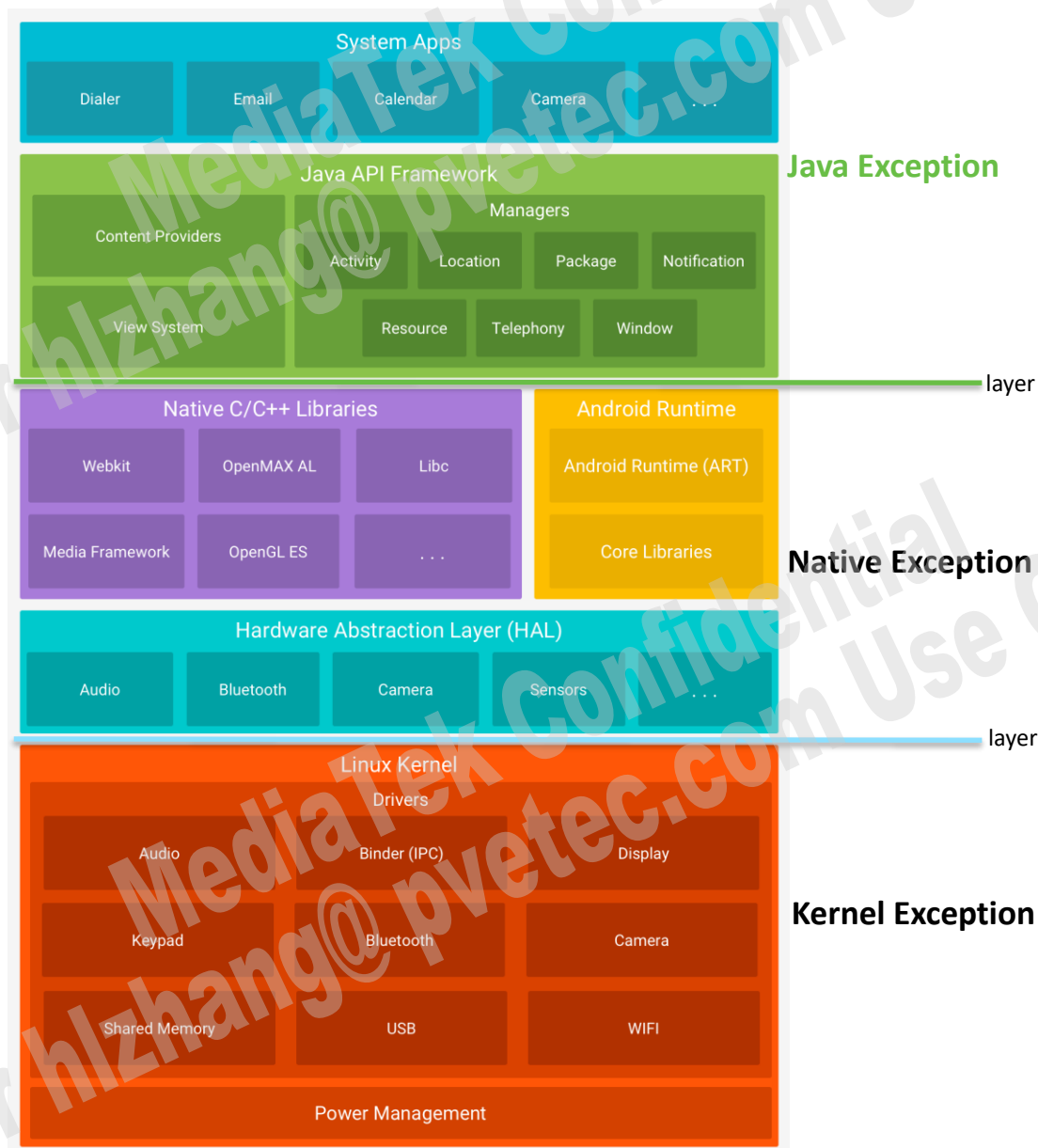


Figure 1-8. Exception classification

1.5.2.5 GAT Analyzes DB

1. Open the GAT tool.
2. Select toolbar Window→Open DBviewer.

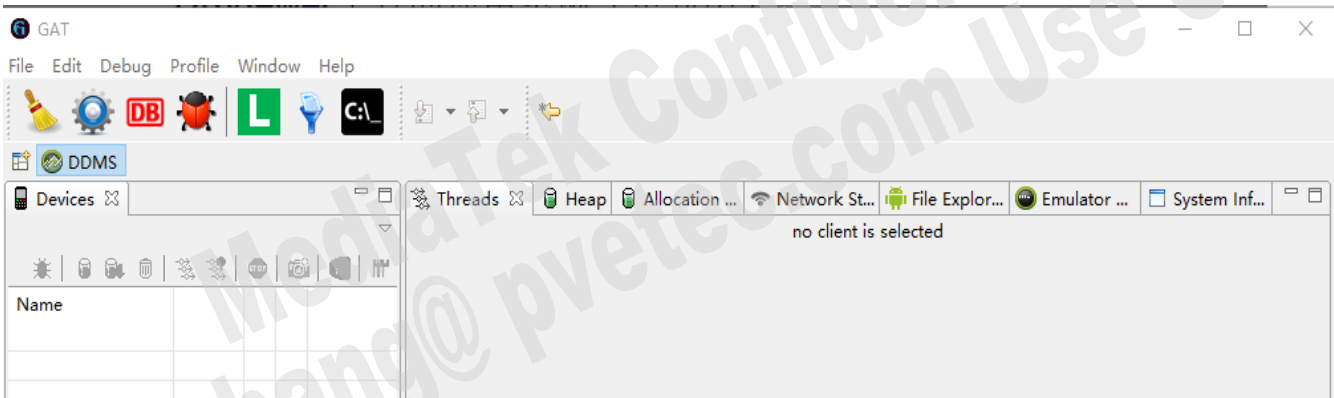


Figure 1-9. GAT tool UI

3. DBViewer UI is shown as Figure 1-10.

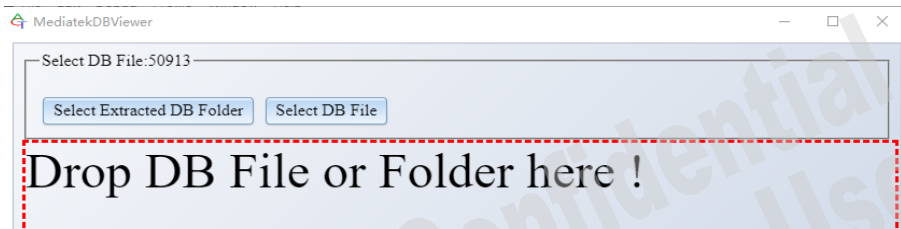


Figure 1-10. DBViewer UI

4. Select DB File as db.fatal.00.KE.dbg. After clicking Start, you can see the parsed DB files after the parsing is completed. The corresponding folder can be found in the same path as the dbg file.

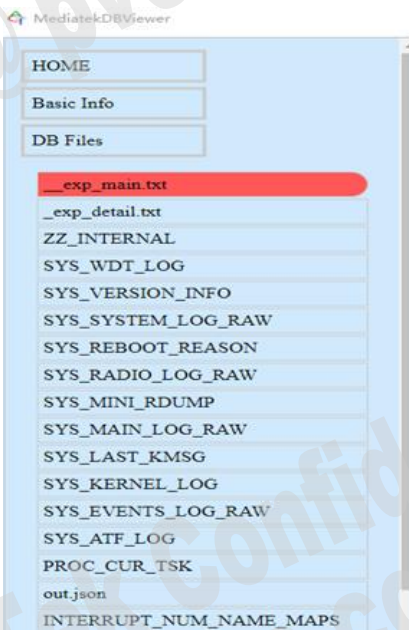


Figure 1-11. Generated file after parsing

1.5.2.6 KE Example

- KE: Kernel Exception
- KE DB file: db.fatal.00.KE. When a KE occurs, a DB file is typically generated in /data/vendor/aee_exp.
- Decompressing db.fatal.00.KE.dbg to obtain db.fatal.00.KE.DBG.DEC.
- Unpack the content of the DB and pay particular attention to _exp_main.txt. The SYS_KERNEL_LOG contains the most information, including some kernel logs before the occurrence of an exception and "SYS_LAST_KMSG" logs the final messages before a reboot.

 _exp_main.txt	2020/4/14 17:19
 INTERRUPT_NUM_NAME_MAPS	2020/4/14 17:19
 out.json	2020/4/14 17:19
 PROC_CUR_TSK	2020/4/14 17:19
 SYS_KERNEL_LOG_RAW	2020/4/14 17:19
 SYS_LAST_KMSG	2020/4/14 17:19
 SYS_MINI_RDUMP	2020/4/14 17:19
 SYS_RAMCONSOLE_RAW	2020/4/14 17:19
 SYS_REBOOT_REASON	2020/4/14 17:19
 SYS_VERSION_INFO	2020/4/14 17:19
 ZAEE_LOG	2020/4/14 17:19
 ZZ_INTERNAL	2020/4/14 17:19

Figure 1-12. Files generated after decompression

- From _exp_main.txt you can see this issue is caused by the USBNET module.

```
Exception Class: Kernel (KE)
Exception Type: WDT timeout and Hardware reboot
PC is at [<ffffffc0007e7948>] skb_put+0xe4/0x108

Current Executing Process:
[khubd, 50] [kthreadd, 2]

Backtrace:
[<ffffffc0009658f0>] __do_kernel_fault.part.5+0x70/0x84
[<ffffffc0000942d4>] do_bad_area+0x90/0x94
[<ffffffc000094320>] do_translation_fault+0x30/0x4c
[<ffffffc0000813f8>] do_mem_abort+0x38/0x98
[<ffffffc000083c58>] ell_da+0x1c/0x88
[<ffffffc00067ec64>] rx_complete+0x28/0x1ec
[<ffffffc000688d7c>] __usb_hcd_giveback_urb+0x58/0xf4
[<ffffffc000689448>] usb_hcd_giveback_urb+0x38/0x104
[<ffffffc00061de98>] musb_giveback+0xb8/0x12c
[<ffffffc00062373c>] musb_advance_schedule+0x270/0x754
[<ffffffc000625138>] musb_cleanup_urb.isra.9+0x118/0x524
[<ffffffc000625d18>] musb_urb_dequeue+0x3a8/0x3b4
[<ffffffc000689e94>] usb_hcd_unlink_urb+0x74/0xd8
[<ffffffc00068b930>] usb_unlink_urb+0x20/0x44
[<ffffffc00067d4c4>] unlink_urbs.isra.17+0x9c/0xc8
[<ffffffc00067d5ac>] usbnet_terminate_urbs+0x7c/0x11c
[<ffffffc00067de14>] usbnet_stop+0xe0/0x15c
```

Figure 1-13. USBNET causing KE

- For the KE issue, some debug information that needs to be provided is shown in Table 1-3:

Table 1-3. KE log map

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
KE	M	C	M	M	X	X	X

vmlinux Symbol path: out/target/product/\$proj/obj/KERNEL_OBJ/vmlinux

1.5.2.7 SWT Example

SWT: Software Watchdog Timeout

SWT DB file: db.fatal.01.SWT

Decompress db.fatal.01.SWT to obtain db.fatal.01.SWT.dbg.DEC.

Unpack the content of the DB and pay particular attention to _exp_main.txt.

 _exp_main.txt	2020/12/5 11:09
 CMDQ_RECORD	2020/12/5 11:09
 CMDQ_STATUS	2020/12/5 11:09
 DUMPSYS_MEMINFO	2020/12/5 11:09
 DUMPSYS_SURFACEFLINGER	2020/12/5 11:09
 out.json	2020/12/5 11:09
 PROCESS_CMDLINE	2020/12/5 11:09
 PROCESS_ENVIRONMENT	2020/12/5 11:09
 PROCESS_FILE_STATE	2020/12/5 11:09
 PROCESS_MAPS	2020/12/5 11:09
 PROCESS_OOM_ADJ	2020/12/5 11:09
 PROCESS_OOM_SCORE	2020/12/5 11:09
 PROCESS_SCHED	2020/12/5 11:09
 PROCESS_SHOWMAP	2020/12/5 11:09
 PROCESS STATE	2020/12/5 11:09

Figure 1-14. SWT DB files generated after decompression

- From exp_main.txt you can see an exception occurs in system_server, which leads to SWT.

```

$** ** Fatal **
Build Info: 'L1.MP13:B200_V1.0.0_320.240_1_V0.0.2_20181128_R:mt6735:S01
Flavor Info: 'None'
Exception Log Time:[Thu Dec 3 14:54:21 CST 2020] [128.466366]

Exception Class: SWT
Exception Type: system_server_watchdog

Current Executing Process:
system_server

Backtrace:
Process: system_server
Subject: surfaceflinger hang.
Build: H_DiDi_B200_V3.1_20201202_R

$** ** Fatal **

```

Figure 1-15. system_server causing SWT

1.5.2.8 NE Example

- NE: Native Exception
- NE DB: db.fatal.NE
- Decompress db.fatal.00.NE.dbg to obtain db.fatal.00.NE.dbg.DEC.
- Unpack the content of the DB and pay particular attention to _exp_main.txt.

 _exp_main.txt	2020/12/24 16:34
 _exp_detail.txt	2020/12/24 16:34
 LSHAL_CAMERA	2020/12/24 16:34
 out.json	2020/12/24 16:34
 PROCESS_ATTR	2020/12/24 16:34
 PROCESS_CMDLINE	2020/12/24 16:34
 PROCESS_COREDUMP	2020/12/24 16:34
 PROCESS_ENVIRONMENT	2020/12/24 16:34
 PROCESS_FILE_STATE	2020/12/24 16:34
 PROCESS_MAPS	2020/12/24 16:34

Figure 1-16. NE DB file gen

- From exp_main.txt you can see systemui exception causes NE.

Exception Class: Native (NE)
Exception Type: SIGABRT

Current Executing Process:
pid: 1271, tid: 1271
com.android.systemui

Backtrace:

```
#00 pc 00000000000821a0 /apex/com.android.runtime/lib64/bionic/libc.so (abort+176)
#01 pc 0000000000004ff4 /system/bin/app_process64 (art::SignalChain::Handler(int, siginfo*, void*)+1132)
#02 pc 0000000000000638 [vdso:00000072cab80000] (__kernel_rt_sigreturn)
#03 pc 000000000000cd6a4 /apex/com.android.runtime/lib64/bionic/libc.so (__epoll_pwait+4)
#04 pc 0000000000017da8 /system/lib64/libutils.so (android::Looper::pollInner(int)+144)
#05 pc 0000000000017c78 /system/lib64/libutils.so (android::Looper::pollOnce(int, int*, int*, void**)+56)
#06 pc 00000000001392bc /system/lib64/libandroid_runtime.so (android::android_os_MessageQueue_nativePoll0
#07 pc 000000000002d7afc /system/framework/arm64/boot-framework.oat (art_jni_trampoline+140)
#08 pc 00000000007f9e04 /system/framework/arm64/boot-framework.oat (android.os.MessageQueue.next+228)
#09 pc 00000000007f6c08 /system/framework/arm64/boot-framework.oat (android.os.Looper.loop+680)
#10 pc 000000000057dlcc /system/framework/arm64/boot-framework.oat (android.app.ActivityThread.main+812)
#11 pc 00000000001365b8 /apex/com.android.runtime/lib64/libart.so (art_quick_invoke_static_stub+568)
#12 pc 00000000001450cc /apex/com.android.runtime/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, u
#13 pc 00000000004b1190 /apex/com.android.runtime/lib64/libart.so (art::(anonymous namespace)::InvokeWith
#14 pc 00000000004b2bd8 /apex/com.android.runtime/lib64/libart.so (art::InvokeMethod(art::ScopedObjectAcc
#15 pc 000000000043e928 /apex/com.android.runtime/lib64/libart.so (art::Method::Invoke(JNIEnv*, jobject*)
#16 pc 00000000000c5c34 /system/framework/arm64/boot.oat (art_jni_trampoline+180)
#17 pc 0000000000a78218 /system/framework/arm64/boot-framework.oat (com.android.internal.os.RuntimeInit$M
#18 pc 0000000000a801e4 /system/framework/arm64/boot-framework.oat (com.android.internal.os.ZygoteInit.ma
#19 pc 00000000001365b8 /apex/com.android.runtime/lib64/libart.so (art_quick_invoke_static_stub+568)
#20 pc 00000000001450cc /apex/com.android.runtime/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, u
#21 pc 00000000004b1190 /apex/com.android.runtime/lib64/libart.so (art::(anonymous namespace)::InvokeWith
#22 pc 00000000004b0df0 /apex/com.android.runtime/lib64/libart.so (art::InvokeWithVarArgs(art::ScopedObjectAcc
#23 pc 00000000003bba30 /apex/com.android.runtime/lib64/libart.so (art::JNI::CallStaticVoidMethodV(JNIEnv
#24 pc 00000000000c058c /system/lib64/libandroid_runtime.so (JNIEnv::CallStaticVoidMethod(jclass*, jme
#25 pc 00000000000c34ac /system/lib64/libandroid_runtime.so (android::AndroidRuntime::start(char const*,
#26 pc 0000000000034e0 /system/bin/app_process64 (main+1168)
#27 pc 000000000007d86c /apex/com.android.runtime/lib64/bionic/libc.so (__libc_init+108)
```

*** Exception ***

Figure 1-17. Systemui causing NE

- For NE issue, the following logs should be provided:

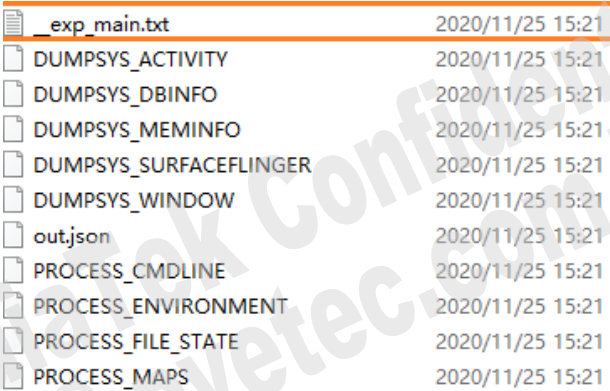
Table 1-4. NE log map

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
NE	M	X	M	M	X	X	X

NE Symbol Path: out/target/product/\$proj/symbols

1.5.2.9 JE Example

- JE: Java Exception
- JE DB: db.fatal.00.JE
- Decompress db.fatal.00.JE.dbg to obtain db.fatal.00.JE.dbg.DEC.
- Unpack the content of the DB and pay particular attention to _exp_main.txt.



 _exp_main.txt	2020/11/25 15:21
 DUMPSYS_ACTIVITY	2020/11/25 15:21
 DUMPSYS_DBINFO	2020/11/25 15:21
 DUMPSYS_MEMINFO	2020/11/25 15:21
 DUMPSYS_SURFACEFLINGER	2020/11/25 15:21
 DUMPSYS_WINDOW	2020/11/25 15:21
 out.json	2020/11/25 15:21
 PROCESS_CMDLINE	2020/11/25 15:21
 PROCESS_ENVIRONMENT	2020/11/25 15:21
 PROCESS_FILE_STATE	2020/11/25 15:21
 PROCESS_MAPS	2020/11/25 15:21

Figure 1-18. JE DB file generated after decompression

- From exp_main.txt you can see animation exception causes system_server crash and leads to JE.

Exception Class: Java (JE)
Exception Type: system_server_crash

Current Executing Process:
system_server

Backtrace:

Process: system_server

Build: NEUSOFT/full_fs1lg_qj_2wk/fs1lg_qj_2wk:5.1/LMY47D/1605154054:user/test-keys

```
java.lang.IllegalArgumentException: The Path cannot have discontinuity in the X axis.
    at android.view.animation.PathInterpolator.initPath(PathInterpolator.java:177)
    at android.view.animation.PathInterpolator.initCubic(PathInterpolator.java:154)
    at android.view.animation.PathInterpolator.parseInterpolatorFromTypeArray(PathInterpolat
    at android.view.animation.PathInterpolator.<init>(PathInterpolator.java:100)
    at android.view.animation.AnimationUtils.createInterpolatorFromXml(AnimationUtils.java:
    at android.view.animation.AnimationUtils.loadInterpolator(AnimationUtils.java:283)
    at com.android.internal.policy.impl.PhoneWindow$DecorView.<init>(PhoneWindow.java:2331)
    at com.android.internal.policy.impl.PhoneWindow.generateDecor(PhoneWindow.java:3465)
    at com.android.internal.policy.impl.PhoneWindow.installDecor(PhoneWindow.java:3851)
    at com.android.internal.policy.impl.PhoneWindow.setContentView(PhoneWindow.java:411)
    at com.android.internal.app.AlertController.installContent(AlertController.java:240)
    at android.app.AlertDialog.onCreate(AlertDialog.java:356)
    at android.app.ProgressDialog.onCreate(ProgressDialog.java:198)
    at android.app.Dialog.dispatchOnCreate(Dialog.java:386)
    at android.app.Dialog.show(Dialog.java:287)
```

Figure 1-19. Animation causing system_server crash and leading to JE

1.5.2.10 EE Example

- EE: External Exception, such as modem, connectivity exception.
- EE DB: db.02.EE
- Decompress db.02.EE.dbg to obtain db.02.EE.dbg.DEC.
- Unpack the content of the DB and pay particular attention to _exp_main.txt.
- MD encountered an assert, as shown in [Figure 1-20](#).

```
Exception Log Time:[Sat Dec 6 13:35:09 CST 2023] [11633.650000]
Exception Class: External (EE)
Exception Type: modem
Current Executing Process:
mdl:
[ASSERT] file:RXAGC_C1_Main.c @FMC line:867
```

Figure 1-20. MD ASSERT triggered

1.5.2.11 ANR Example

- ANR: Application Not Response. A certain App's main thread did not finish a certain task within a period of time.
- ANR DB: db.00.ANR
- Decompress db.00.ANR.dbg to obtain db.00.ANR.dbg.DEC.
- Unpack the content of the DB and pay particular attention to _exp_main.txt and SWT_JBT_TRACES.
- From _exp_main.txt you can see ANR occurs in system UI.

```
Exception Class: ANR
Exception Type: system_app_anr
```

```
Current Executing Process:
com.android.systemui
com.android.systemui v22 (5.1-1605618285)
```

```
Backtrace:
Process: com.android.systemui
Flags: 0x40c83e0d
Package: com.android.systemui v22 (5.1-1605618285)
Subject: Broadcast of Intent { act=android.intent.action.TIME_TICK flg=0x50000114 (has extras) }
```

Figure 1-21. ANR occurs in system UI

- For ANR issue, the following debug log should be provided.

Table 1-5. ANR issue log map

Issue Type	Mobile Log	UART Log	db	Symbol	Thermal Log	Top	Ftrace
ANR	M	X	M	O	C	C	C

- How to analyze ANR issue:

The ANR can be quickly located, and essential information can be obtained by checking the androidlog. A helpful tip is to search for “ANR ” with an extra space to quickly find the location and filter out some distracting log information.

```
11-22 15:31:27.520 1785 1800 E ActivityManager: ANR in com.amazon.aiv.eu (com.amazon.aiv.eu/com.amazon.ignition.IgnitionActivity)
11-22 15:31:27.520 1785 1800 E ActivityManager: PID: 6229
11-22 15:31:27.520 1785 1800 E ActivityManager: Reason: Input dispatching timed out (Waiting because no window has focus but there is a f
11-22 15:31:27.520 1785 1800 E ActivityManager: Load: 32.24 / 31.91 / 31.88
11-22 15:31:27.520 1785 1800 E ActivityManager: CPU usage from 0ms to 11679ms later (2017-11-22 15:31:15.731 to 2017-11-22 15:31:27.411):
11-22 15:31:27.520 1785 1800 E ActivityManager: 45% 6229/com.amazon.aiv.eu: 34% user + 10% kernel / faults: 48921 minor 943 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 26% 1201/surfaceflinger: 8.1% user + 18% kernel / faults: 1156 minor 13 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 23% 1785/system_server: 14% user + 8.6% kernel / faults: 9885 minor 487 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 18% 1290/VDEC 0: 0% user + 18% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 13% 650/mmcqd/0: 0% user + 13% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 12% 430/kswapd0: 0% user + 12% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 9.9% 1465/dtv_svc: 3.2% user + 6.6% kernel / faults: 1717 minor 67 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 8.2% 2684/com.mediatek.tvinput: 6% user + 2.1% kernel / faults: 2755 minor 25 major
11-22 15:31:27.520 1785 1800 E ActivityManager: 7.1% 11637/kworker/u9:8: 0% user + 7.1% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 6.9% 1595/MLVdo_thread: 0% user + 6.9% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 6.7% 30988/kworker/u9:0: 0% user + 6.7% kernel
11-22 15:31:27.520 1785 1800 E ActivityManager: 6.6% 2968/org.droidtv.playtv: 5% user + 1.6% kernel / faults: 3315 minor 62 major
```

Figure 1-22. How to quickly locate ANR issues through logs

When an Application Not Responding (ANR) occurs, the corresponding application will receive a SIGQUIT signal for abnormal termination. Subsequently, the Dalvik virtual machine will automatically generate a trace.txt file in the /data/anr/ directory. This file records the execution status of various threads in the system at the time of ANR occurrence.

1.5.3 Memory Leak

1.5.3.1 Brief Introduction

The key information for determining whether there is a memory leak is whether the memory usage keeps increasing.

The cause of a Memory Leak is when a piece of allocated memory cannot be referenced or released, the allocated memory block becomes like an orphaned memory, unable to be used by the system again.

1.5.3.2 Query Memory Command

- `dumpsys meminfo`: View the detailed memory distribution.
- `cat /proc/meminfo`: View the total memory size.

By using a command or a batch script, memory information can be continuously dumped in the background while duplicating phenomenon.

1.5.3.3 Kernel Memory Leak

Command: `cat /proc/meminfo`

```
MemTotal:          965616 kB
MemFree:           114640 kB
MemAvailable:      491072 kB
Buffers:           7224 kB
Cached:           393032 kB
Shmem:             1184 kB
Slab:              55916 kB
SReclaimable:      18788 kB
SUnreclaim:        37128 kB
KernelStack:       12592 kB
PageTables:        17876 kB
VmallocUsed:        0 kB
```

Figure 1-23. `cat /proc/meminfo` log

- If the value of “sunreclaim” is significantly large and continues to increase, it can be inferred that a kernel Memory Leak has occurred, as shown in Figure 1-23.
- If an Out of Memory (OOM) event occurs in the kernel log, this portion of memory will also be dumped. If a significant and sustained increase in “sunreclaim” is observed, it can generally be inferred that there is a kernel Memory Leak, as shown in Figure 1-24.

```
[147842.604198] slab_reclaimable:12762 slab_unreclaimable:119398
.....
[151361.050385] slab_reclaimable:8474 slab_unreclaimable:122915
.....
[153699.412832] slab_reclaimable:7833 slab_unreclaimable:124784
```

Figure 1-24. Kernel memory leak log

1.5.4 WDT Brief Introduction

The WDT stands for watchdog timer. It is a watchdog module, which essentially is a counter that can be reset within a certain period of time. When the watchdog is activated, the counter initiates automatic counting. After a certain period of

time, if the count is not reset, and when the counter reaches a specified value, it will emit a reset signal. Many devices, including the Central Processing Unit (CPU), receive this signal and undergo a reset or restart (commonly known as “being bitten by the dog”). In order to ensure that the watchdog does not issue a reset signal, it is necessary for the watchdog counter to be cleared (commonly known as “feeding the dog”) within the allowed time interval, causing the counter to reset. If the system operates normally and ensures timely “feeding of the dog”, then all is well. Once a program malfunctions and freezes without “feeding the dog”, the system is reset by being “bitten”.

1.5.4.1 SWT Brief Introduction

The SystemServer process is a core process in Android, providing essential services for running apps. If some core service and key threads get stuck in SystemServer, it will lead to corresponding functional abnormalities. For example, if the platform hangs, the input does not respond, the App cannot be started, and other abnormal situations will occur. If there is no mechanism to reset these services, the customer experience will be seriously affected. Therefore, it is necessary to give the system a chance to automatically reset when the core services and key threads are stuck. Therefore, Google introduced the SystemServer watchdog mechanism. This mechanism monitors whether core services and core threads are stuck.

1.5.4.2 HWT Brief Introduction

The WDT phase 1 timeout will trigger FIQ. At this time, the system receives the FIQ and processes it. Finally, it calls BUG() to go through the normal kernel exception process. The generated DB type is Hardware Watchdog Timeout (HWT).

1.5.5 Performance Mode

1.5.5.1 Brief Introduction

Performance mode is a configuration option that improves system performance on devices such as mobile devices, laptops, and desktops. In performance mode, the device prioritizes higher CPU frequency, larger memory cache, higher screen refresh rate, higher power consumption to provide better system performance and responsiveness. This mode is usually used when high performance is required.

1.5.5.2 Performance Mode Setting

For different ICs, the node paths may be different, but the setting method is the same.

CPU:

```
ls /sys/devices/system/cpu/cpufreq/ //show how many clusters this IC has
echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy7/scaling_governor
```

or:

```
adb shell "echo 0 1900000 1900000 > /proc/cpudvfs/cpufreq_debug"
adb shell "echo 4 2800000 2800000 > /proc/cpudvfs/cpufreq_debug"
adb shell "echo 7 2900000 2900000 > /proc/cpudvfs/cpufreq_debug"
```

first number (1900000) means floor and second number means ceiling.

GPU Set:

```
adb shell "echo 0 > /proc/gpufreqv2/fix_target_opp_index"
```

GPU Query:

```
adb shell "cat /proc/gpufreqv2/gpufreq_status"
```

DDR:

Set DDR frequency.

```
adb shell "echo 0 > /sys/kernel/helio-dvfsrc/dvfsrc_force_vcore_dvfs_opp"
```

Check DDR frequency.

```
adb shell "cat /sys/kernel/helio-dvfsrc/dvfsrc_dump | grep bps"
```

Thermal disable:

```
adb shell "thermal_intf apply disable_throttling.conf"
```

1.5.5.3 Performance Mode Auto Started

Perform the following modification in device/mediatek/mtxx/init.mtxxx.rc:

```
# switch to sched-dvfs
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor "schedutil"
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor "schedutil"
write /sys/class/net/p2p0/queues/rx-0/rps_cpus "fe"
write /sys/class/net/p2p0/queues/rx-1/rps_cpus "fe"
write /sys/class/net/p2p0/queues/rx-2/rps_cpus "fe"
write /sys/class/net/p2p0/queues/rx-3/rps_cpus "fe"
# stop bootprof
write /proc/bootprof 0
```

Modify these two lines:

```
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor "schedutil"
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor "schedutil"
```

to:

```
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor "performance"
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor "performance"
```

1.5.6 CPU Hotplug

Turn on/off CPU by this command as below:

```
adb shell "echo 1 > /sys/devices/system/cpu/cpu1/online"
adb shell "echo 1 > /sys/devices/system/cpu/cpu2/online"
adb shell "echo 1 > /sys/devices/system/cpu/cpu3/online"
```

echo 1 will turn ON, echo 0 will turn OFF.

1.5.7 Systrace Capture

1. Use the adb command to capture trace 10s; 10s means capturing for 10s, and the maximum duration is 20s.

```
adb shell perfetto -o /data/misc/perfetto-traces/trace -t 10s sched freq idle am wm gfx
view
```

2. Use script to capture ftrace.

01-catch.bat CPU sched event and workqueue event can be captured.

3. Use Google trace tool after Android Q.

Step1 command: adb shell am start com.android.traceur/com.android.traceur.MainActivity

Step2: Then set the CPU parameters to the maximum and click record trace to start recording the abnormal state.

After a period of time, close it and capture the trace.

Trace storage path: /data/local/traces

4. The trace shows whether the process has entered the queue, whether it has been executed, and which CPU it has been assigned to. If the workqueue is not tied to a core, it can only rely on the system to schedule processes according to loading. The process may run on any CPU.

1.5.8 System Reference FAQ

Memory leak topic: <https://online.mediatek.com/apps/quickstart/QS00086>

Memory corruption topic: <https://online.mediatek.com/apps/quickstart/QS00090>

Hang issue fast analyze: <http://online.mediatek.com/QuickStart/QS00044>

Memory Related: <http://online.mediatek.com/QuickStart/QS00227>

Performance debug guide: <https://online.mediatek.com/apps/quickstart/QS00288>

Advanced Android native exception arch: <http://online.mediatek.com/QuickStart/QS00038>

Advanced HWT arch: <http://online.mediatek.com/QuickStart/QS00067>

Hang Detect fast analyze: <https://online.mediatek.com/apps/quickstart/QS00076>

Advance Linux kernel exception arch: <http://online.mediatek.com/QuickStart/QS00073>

NE/KE training course: <https://online.mediatek.com/apps/quickstart/QS00068>

NE/KE analyze guide: <https://online.mediatek.com/apps/quickstart/QS00089>

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.